Dremel: Interactive Analysis of Web-Scale Datasets

Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer,
Shiva Shivakumar, Matt Tolton, Theo Vassilakis (Google)

VLDB 2010

Speed matters



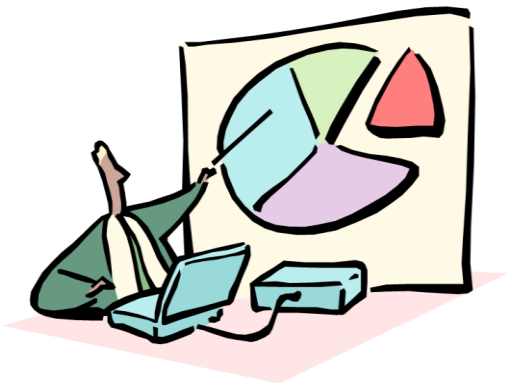
Interactive Tools



Spam



Trends Detection



Web Dashboards



Network Optimization

Dremel system

- Trillion-record, multi-terabyte datasets at interactive speed
  - Scales to thousands of nodes
  - Fault tolerant execution
- Nested data model
  - Complex datasets
  - Columnar storage and processing
- Tree architecture (as in web search)
- Interoperates with Google's data mgmt tools
  - *In situ* data access (e.g., GFS, Bigtable)
  - MapReduce pipelines

Widely used inside Google

- Analysis of crawled web documents
- Tracking install data for applications on Android Market
- Crash reporting for Google products
- OCR results from Google Books
- Spam analysis
- Debugging of map tiles on Google Maps

- Tablet migrations in managed Bigtable instances
- Results of tests run on Google's distributed build system
- Disk I/O statistics for hundreds of thousands of disks
- Resource monitoring for jobs run in Google's data centers
- Symbols and dependencies in Google's codebase

Example: data exploration

1. Runs a MapReduce to extract billions of signals from web pages

2. Ad hoc SQL against Dremel

```
DEFINE TABLE t AS /path/to/data/*
SELECT TOP(signal, 100), COUNT(*) FROM t
. . .
```

3. More MR-based processing on data (FlumeJava [PLDI'10], Sawzall [Sci.Pr.'05])

Data Model

1. Strongly typed nested record

   T = dom | <A1 : T[*|?]..... An : T[*|?]?

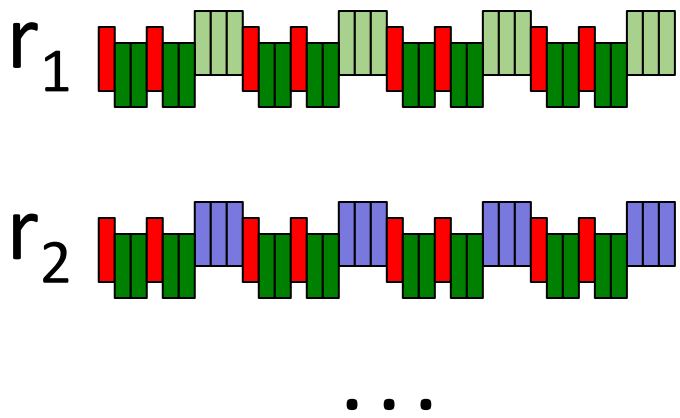
2. Platform neutral and extensible

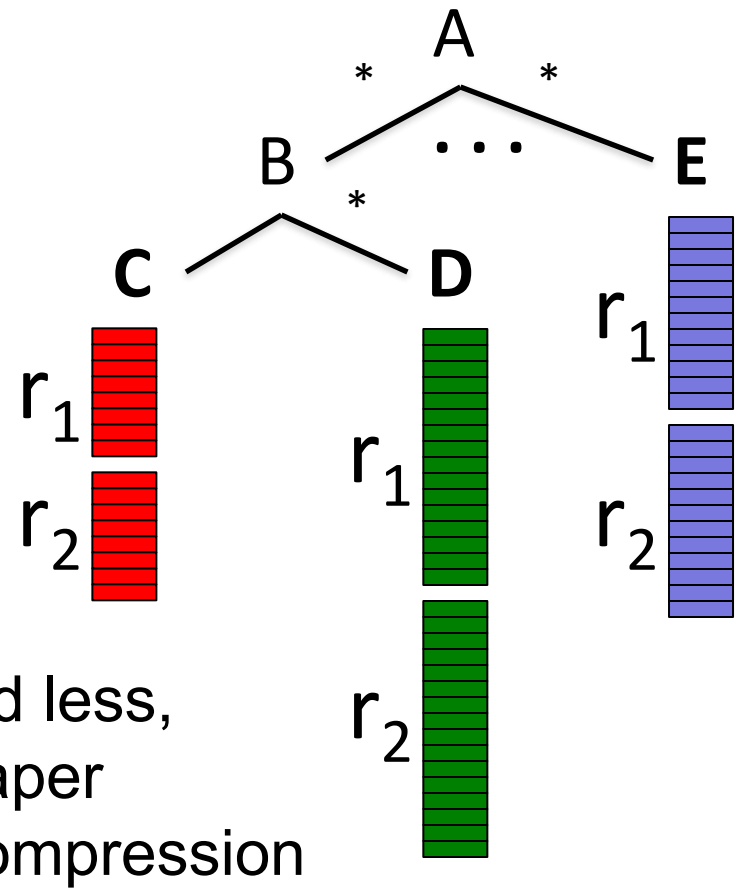
Outline

- Nested columnar storage
- Query processing
- Experiments
- Observations

Records vs. columns

```
DocId: 10                    r1
Links
  Forward: 20
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Url: 'http://A'
Name
  Url: 'http://B'
```





Read less, cheaper decompression

Challenge: preserve structure, reconstruct from a subset of fields

Nested data model

```
message Document {                          multiplicity:
  required int64 DocId;                     [1,1]
  optional group Links {
    repeated int64 Backward;                [0,*]
    repeated int64 Forward;
  }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country;              [0,1]
    }
    optional string Url;
  }
}
```

r_1

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

r_2

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward:  80
Name
  Url: 'http://C'
```

Column-striped representation

DocId

value	r	d
10	0	0
20	0	0

Name.Url

value	r	d
http://A	0	2
http://B	1	2
NULL	1	1
http://C	0	2

Links.Forward

value	r	d
20	0	2
40	1	2
60	1	2
80	0	2

Links.Backward

value	r	d
NULL	0	1
10	0	2
30	1	2

Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

Repetition and definition levels

r=1 r=2 (non-repeating)

Name.Language.Code

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

record (r=0) has repeated

Language (r=2) has repeated

r_1

```
DocId: 10
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

r: At what repeated field in the field's path the value has repeated

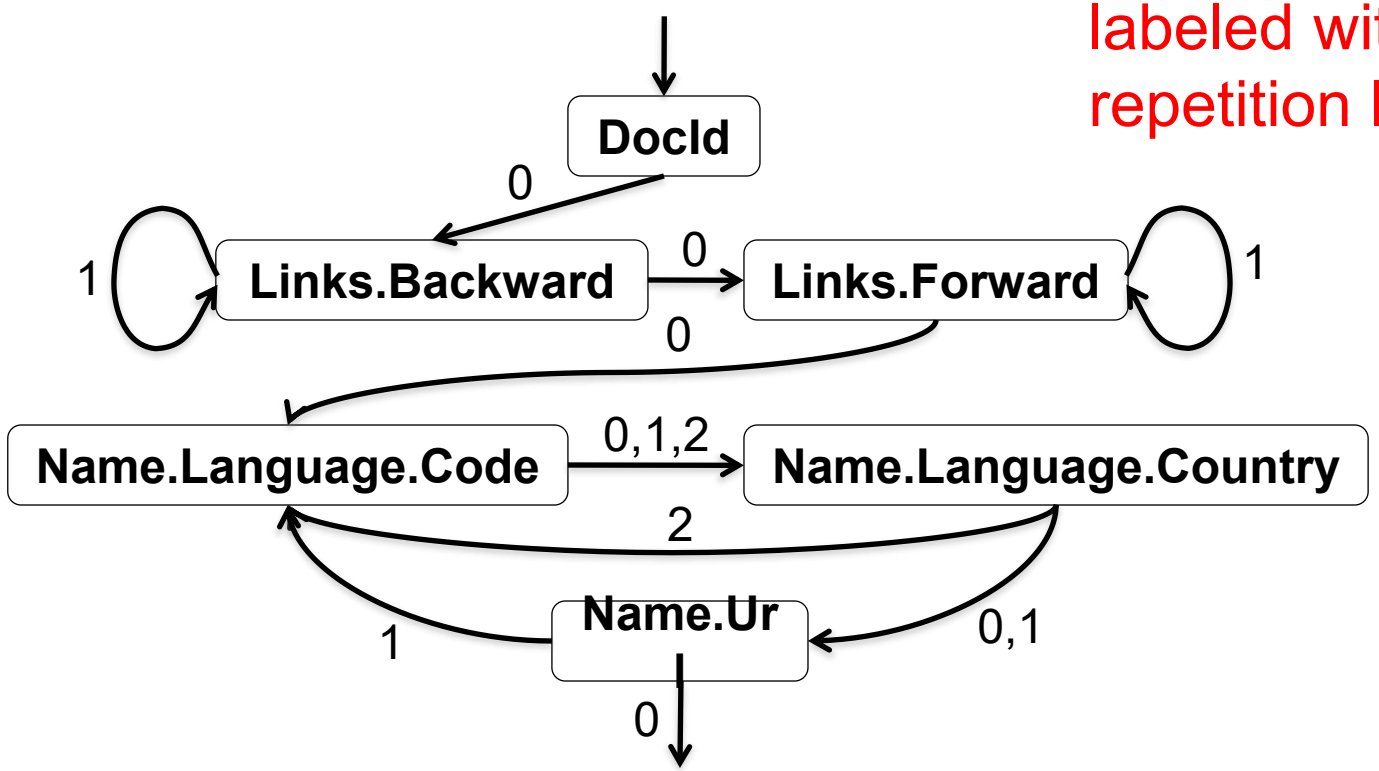
d: How many fields in paths that could be undefined (opt. or rep.) are actually present

r_2

```
DocId: 20
Links
  Backward: 10
  Backward: 30
  Forward:  80
Name
  Url: 'http://C'
```

Record assembly FSM

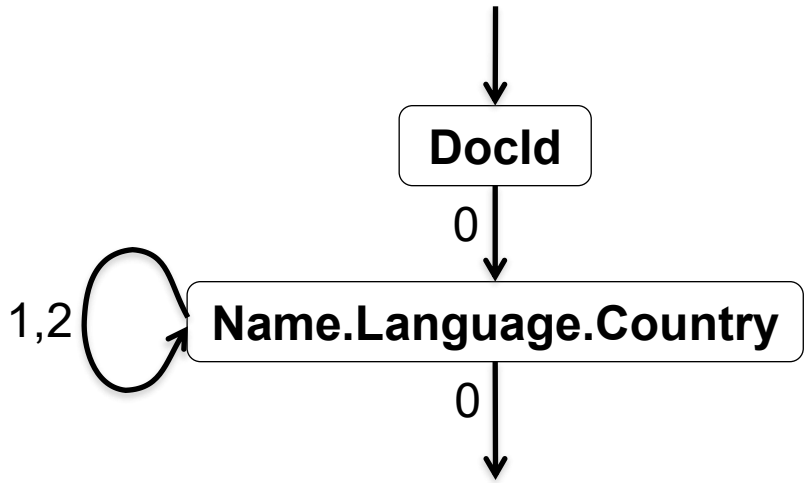
Transitions labeled with repetition levels



For record-oriented data processing (e.g., MapReduce)

Reading two fields

Both Dremel and MapReduce can read same columnar data




```
DocId: 10                    S1
Name
  Language
    Country: 'us'
  Language
Name
Name
  Language
    Country: 'gb'
```

```
DocId: 20                    S2
Name
```

Structure of parent fields is preserved.
Useful for queries like `/Name[3]/Language[1]/Country`

Outline

- Nested columnar storage
- Query processing
- Experiments
- Observations

Query processing

- Optimized for select-project-aggregate
  - Very common class of interactive queries
  - Single scan
  - Within-record and cross-record aggregation
- Approximations: count(distinct), top-k
- Joins, temp tables, UDFs, etc.

SQL dialect for nested data

```
SELECT DocId AS Id,
  COUNT(Name.Language.Code) WITHIN Name AS Cnt,
  Name.Url + ',' + Name.Language.Code AS Str
FROM t
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

Output table

t_1

```
Id: 10
Name
  Cnt: 2
  Language
    Str: 'http://A,en-us'
    Str: 'http://A,en'
Name
  Cnt: 0
```

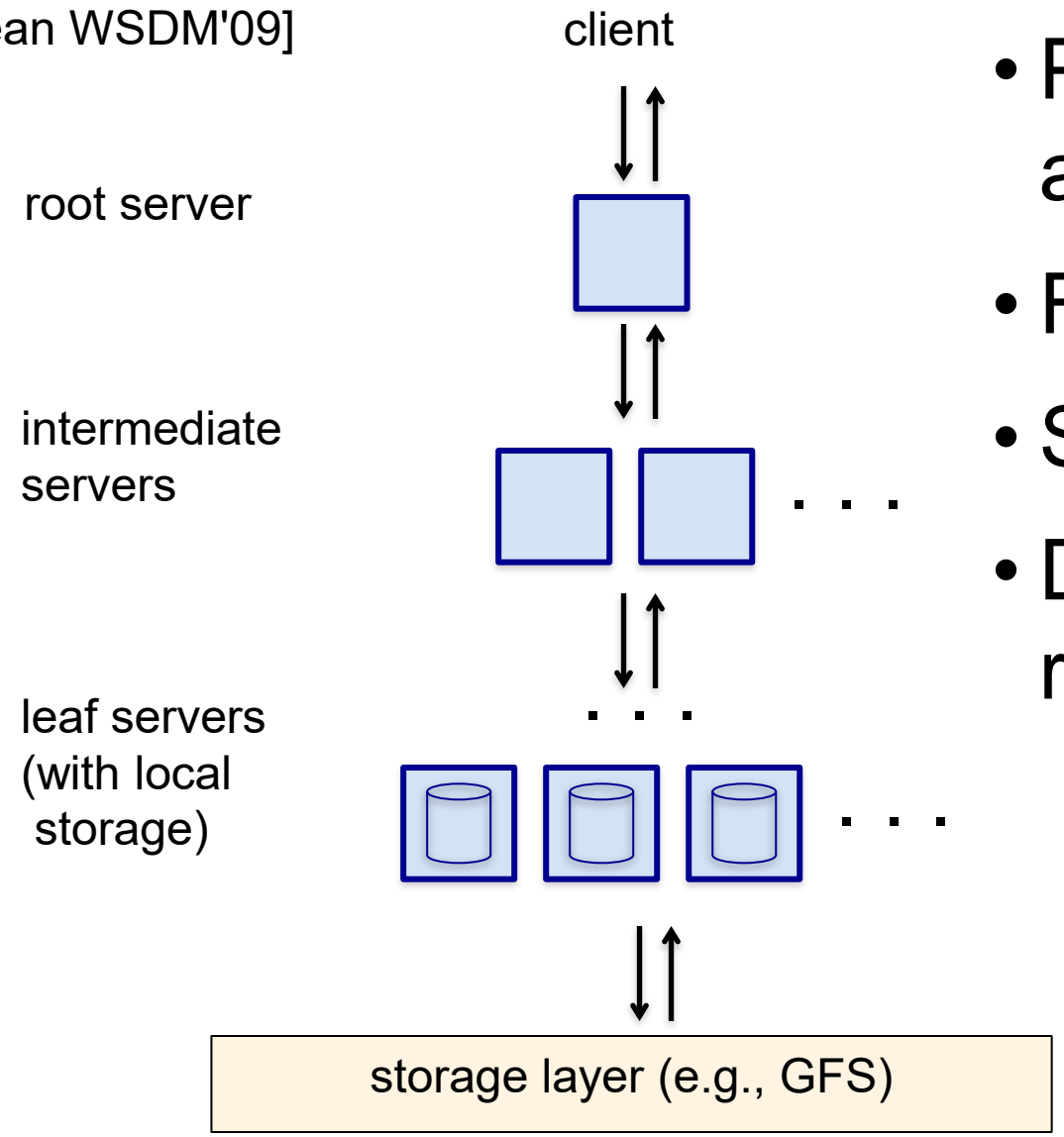
Output schema

```
message QueryResult {
  required int64 Id;
  repeated group Name {
    optional uint64 Cnt;
    repeated group Language {
      optional string Str;
    }
  }
}
```

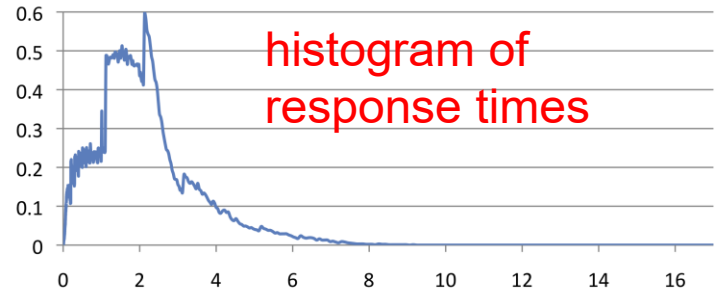
No record assembly during query processing

Serving tree

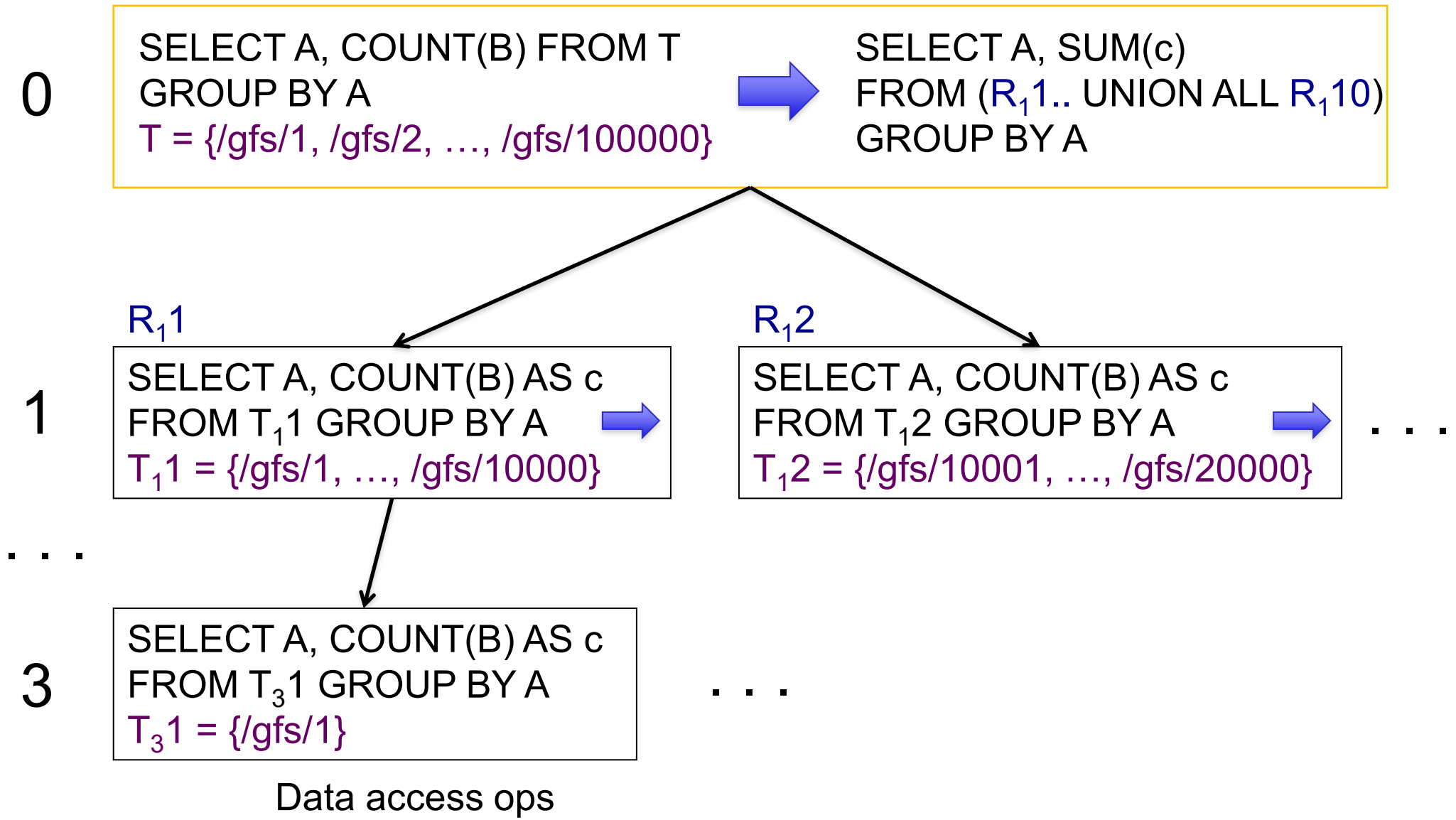
[Dean WSDM'09]



- Parallelizes scheduling and aggregation
- Fault tolerance
- Stragglers
- Designed for "small" results (<1M records)



Example: count()



Outline

- Nested columnar storage
- Query processing
- Experiments
- Observations

Experiments

- 1 PB of real data (uncompressed, non-replicated)
- 100K-800K tablets per table
- Experiments run during business hours

Table name	Number of records	Size (unrepl., compressed)	Number of fields	Data center	Repl. factor
T1	85 billion	87 TB	270	A	3×
T2	24 billion	13 TB	530	A	3×
T3	4 billion	70 TB	1200	A	3×
T4	1+ trillion	105 TB	50	B	3×
T5	1+ trillion	20 TB	30	B	2×

Read from disk

"cold" time on local disk, averaged over 30 runs

2-4x overhead of using records

10x speedup using columnar storage

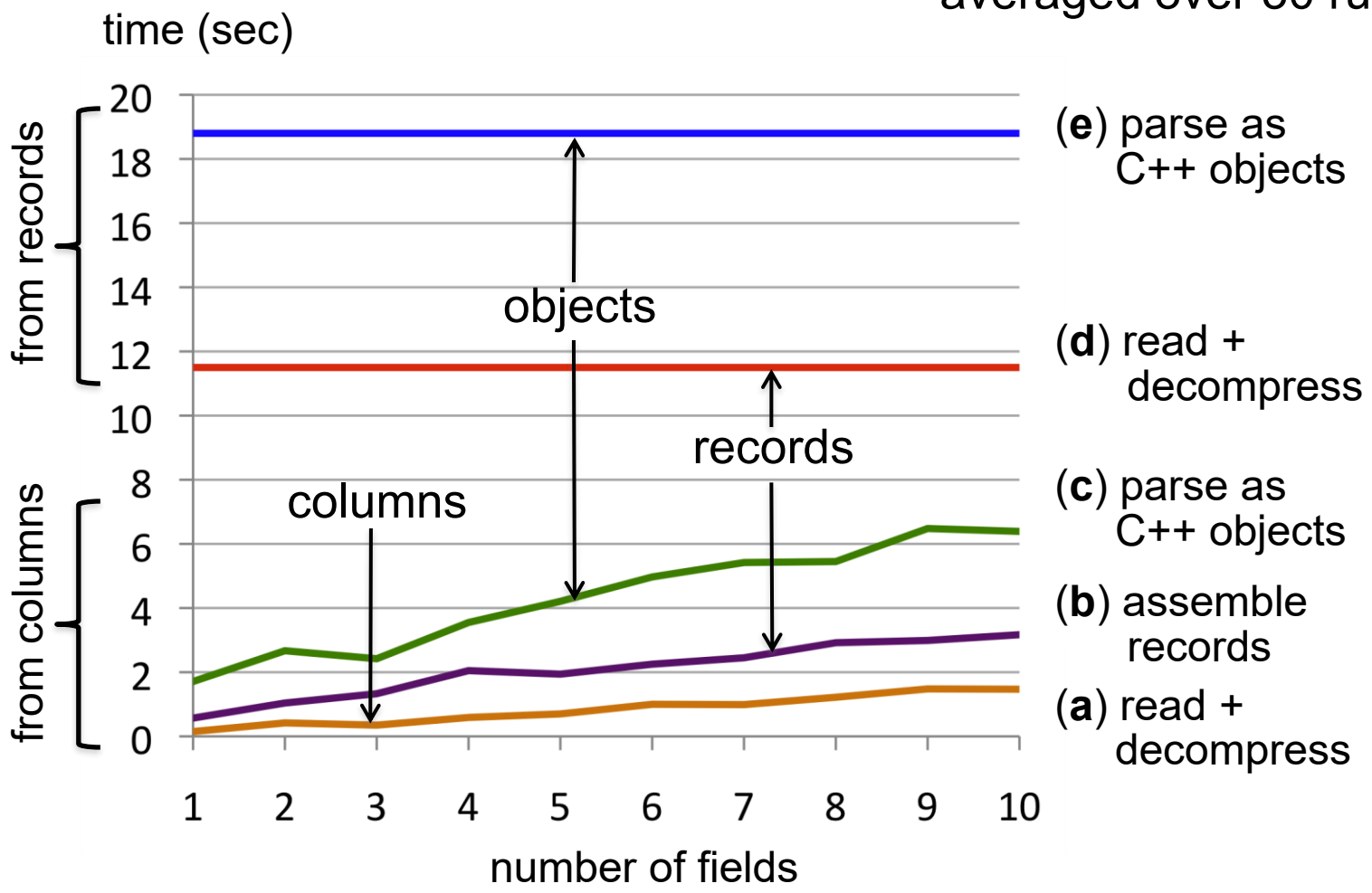
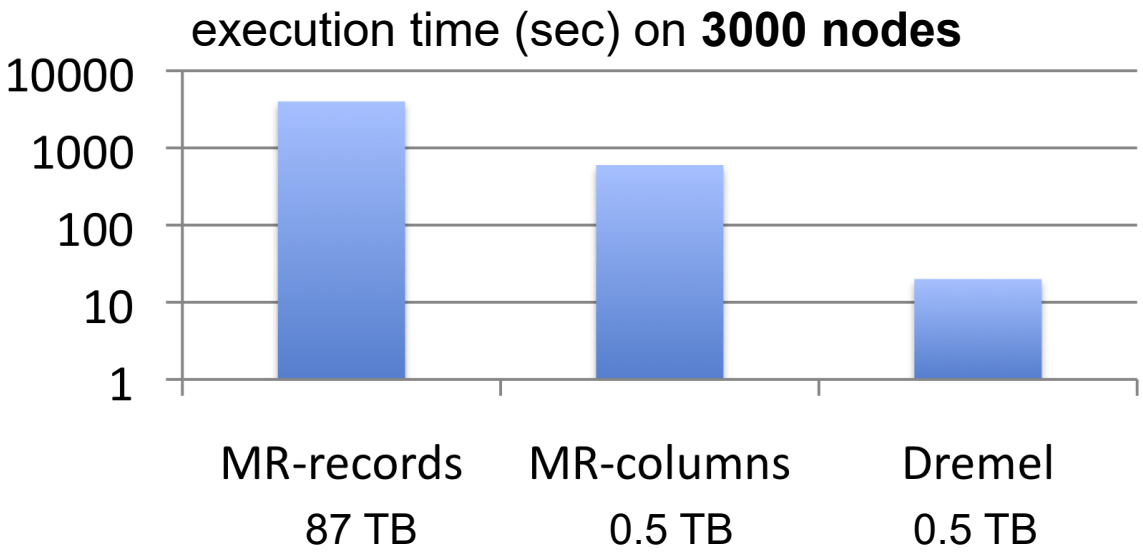


Table partition: 375 MB (compressed), 300K rows, 125 columns

MR and Dremel execution

Avg # of terms in txtField in 85 billion record table T1



Sawzall program ran on MR:

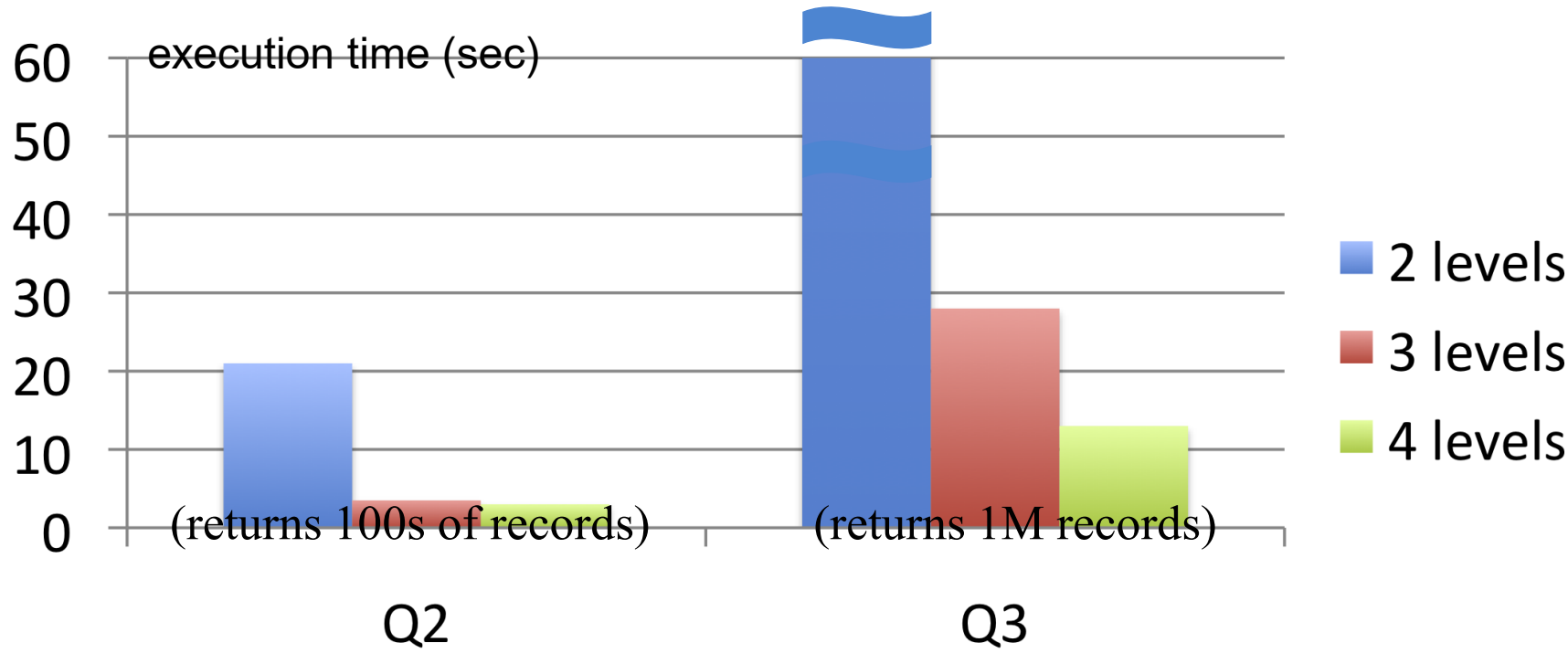
```
num_recs: table sum of int;
num_words: table sum of int;
emit num_recs <- 1;
emit num_words <-
  count_words(input.txtField);
```

Q1:

```
SELECT SUM(count_words(txtField)) / COUNT(*)
FROM T1
```

MR overheads: launch jobs, schedule 0.5M tasks, assemble records

Impact of serving tree depth

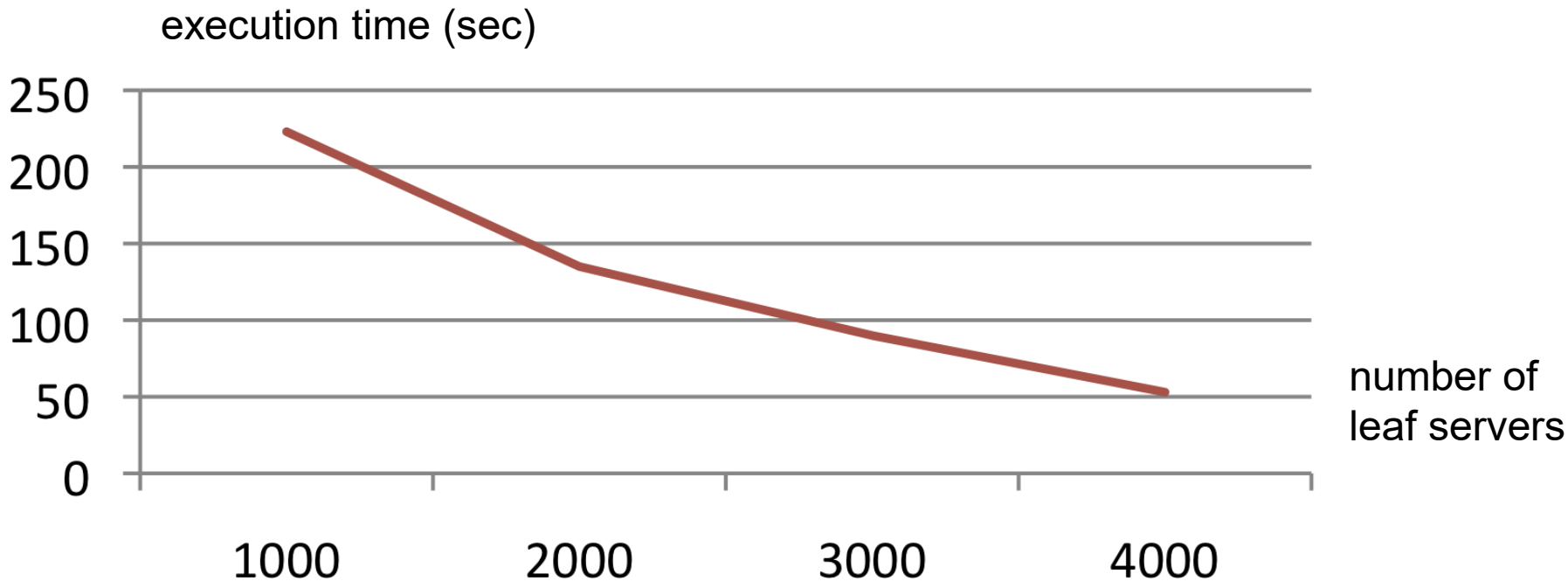


Q2:
```
SELECT country, SUM(item.amount) FROM T2
GROUP BY country
```

Q3:
```
SELECT domain, SUM(item.amount) FROM T2
WHERE domain CONTAINS '.net'
GROUP BY domain
```

40 billion nested items

Scalability



Q5 on a trillion-row table T4:

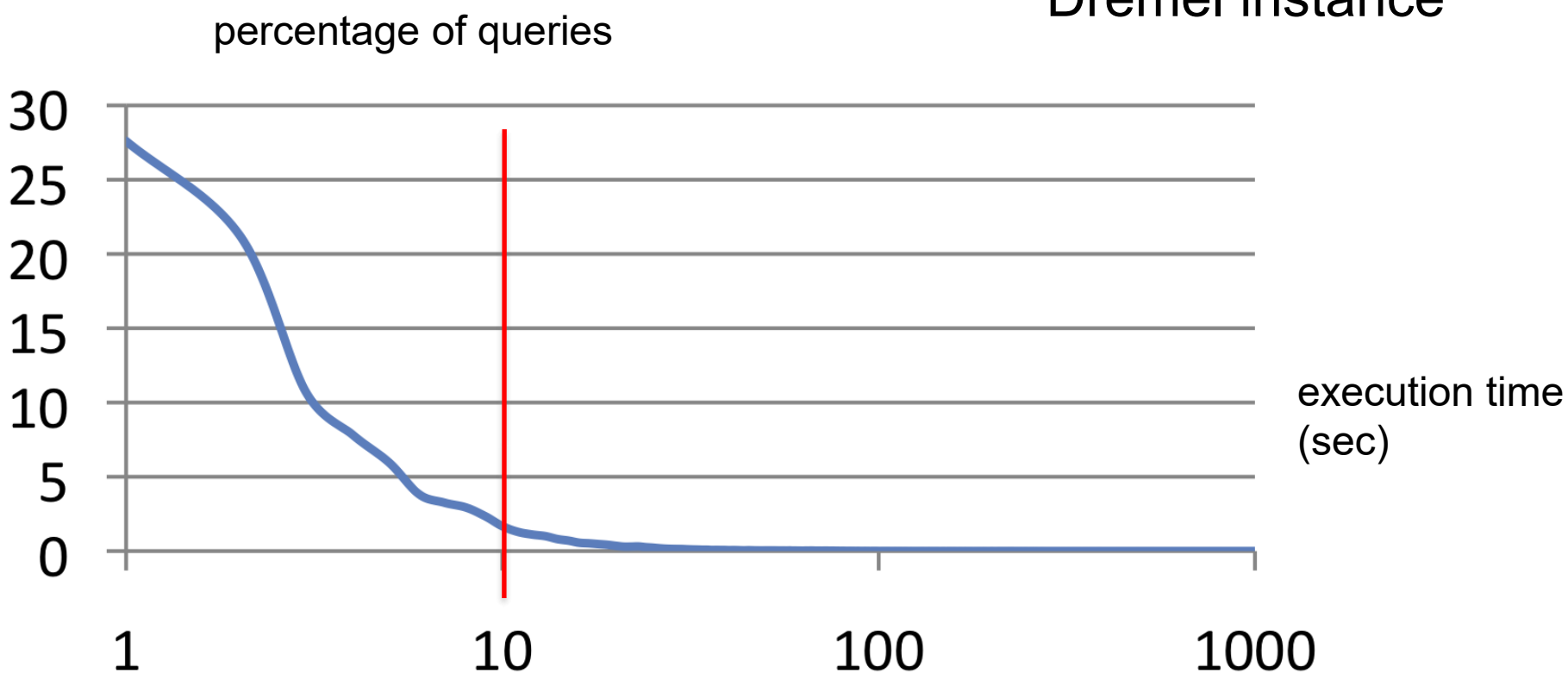
```
SELECT TOP(aid, 20), COUNT(*) FROM T4
```

Outline

- Nested columnar storage
- Query processing
- Experiments
- Observations

Interactive speed

Monthly query workload of one 3000-node Dremel instance



Most queries complete under 10 sec

Observations

- Possible to analyze large disk-resident datasets interactively on commodity hardware
  - 1T records, 1000s of nodes
- MR can benefit from columnar storage just like a parallel DBMS
  - But record assembly is expensive
  - Interactive SQL and MR can be complementary
- Parallel DBMSes may benefit from serving tree architecture just like search engines
- Getting to last few percent of data *fast* is hard
  - Replication, approximation, early termination